CareReach

Find India's real maternal-care gaps — and know which you can trust.

Plain-English guide to the design and how it works

Databricks Apps & Agents for Good Hackathon 2026 — Track 2 (Medical Desert Planner)

1. The problem, in one minute

Across India, planners and NGOs constantly decide where to send scarce maternal-health resources — like a mobile maternal-health unit. They work from messy, web-scraped lists of hospitals and clinics. A listing might claim it performs C-sections, but it could really be a dental clinic. If the planner trusts the wrong listing, a unit goes where it isn't needed, while a community that truly lacks care stays uncovered.

CareReach helps a non-technical planner ask a plain question — “Where in Bihar should we send a mobile maternal-health unit?” — and get a ranked, evidence-backed, honest answer they can save.

2. The big idea: two scores, not one

Most tools answer “where are the gaps?” with a single score. That quietly hides a second, equally important question: can we even trust the gap, or do we simply have no data there? CareReach never mixes those two things. It gives every region TWO independent scores:

- **Care gap** — how underserved the region is (health burden from official survey data, plus how few facilities have verified maternal capability).
- **Evidence confidence** — how much real, verified facility evidence we actually have to back that gap.

Putting them on a simple 2×2 grid makes the decision obvious:

	LOW evidence confidence	**HIGH evidence confidence**
HIGH care gap	DATA-POOR — investigate first (don’t deploy blindly)	REAL desert — act
LOW care gap	—	Adequately served

Example (Bihar, district level): 22 REAL deserts, 15 DATA-POOR, 1 adequately served. Araria looks like the worst desert by health burden — but it has zero facilities on record, so CareReach flags it DATA-POOR (investigate), not a confirmed gap. Saharsa has facilities on record but few with verified obstetric care — a REAL desert to act on.

3. What you see — the app design

CareReach is a single web app, designed so the visuals never get in the way of the data. It uses an India theme (tricolour banner with the Ashoka Chakra, an ambulance, and a faint India-map watermark) kept light so charts stay easy to read.

- The **2×2 quadrant chart** is the main view — real deserts and data-poor regions are different colours, so you see them apart at a glance. Three count cards sit on top (Real / Data-poor / Served).

- **Sidebar controls:** Care domain (specialty), Geography level (state, city, district, pincode), and a state filter.
- **Drill-down:** pick any region to see its two scores, the raw counts behind them, an honesty banner, and the actual facilities — each marked Verified, Claimed-but-unverified, or No claim, with the exact sentence the claim came from.
- **Ask the planner:** type a question, OR speak it in an Indian language (Hindi, Bengali, Tamil, …). The app shows what it heard, the English translation, and a grounded answer.
- **Save / load deployment plans**, with the recommendation attached, so work is never lost.

4. How it works — step by step

Behind the app is a governed data pipeline on Databricks. It moves data through three quality stages (a common “medallion” pattern: bronze → silver → gold).

1. **Bring in the data (Bronze).** Three sources land as-is: ~10,000 India healthcare facilities, the India Post PIN-code directory, and NFHS-5 (an official district health survey). District map shapes come from open geoBoundaries data.
2. **Clean and structure it (Silver).** Survey ‘suppressed’ marks become blanks (not zero), the PIN directory is de-duplicated, and each facility is placed into its district using its map location (with a PIN-code backup). Facilities with no location are flagged ‘inferred’, never dropped.
3. **Read the free text with AI (Silver).** Each facility’s free-text description is read by an AI function that extracts capabilities (does it do obstetrics? C-sections?) — each as a CLAIM with a confidence score, never as fact.
4. **Verify the claims (Silver/Gold).** A second AI step acts as an auditor: it checks each claim against the facility’s profile. A dental or eye hospital that merely scraped a medical word is marked not-credible and does NOT count toward coverage.
5. **Score every region (Gold).** For each region we compute the two scores — care gap and evidence confidence — and assign it to a quadrant. This becomes the region_signals table the app reads.
6. **Answer, show evidence, and save (App).** The app shows the 2×2, lets the planner drill into evidence, answers questions (typed or spoken), and saves plans to a live database (Lakebase).

5. Voice and translation — how it works

A planner can ask by voice in their own Indian language. The flow spans three places — the browser, a speech recognizer, and Databricks — with a clear trust boundary:

```
You speak → [browser captures audio] → [transcribe to native text] → [translate to English on Databricks] → [grounded answer]
```

1. **Pick the language.** The dropdown maps to a language code (Hindi → hi-IN, Bengali → bn-IN, …) so the recognizer knows what to expect.
2. **Capture in the browser.** The mic records your question; nothing leaves the browser until you press Stop.

3. **Transcribe to native text.** The audio is turned into text in the language you spoke. This uses a free speech recognizer; if a clip fails, the app shows a ‘try again’ message rather than going silent.
4. **Translate to English — on Databricks.** The native text is sent to the language model with a clear instruction to translate (not answer) it, and comes back as clean English. Splitting ‘instruction’ from ‘content’ is what stops the model from answering instead of translating.
5. **Show and answer.** The app displays ‘Heard (language): … → English: …’, fills the question box, and feeds the same evidence-grounded planner used everywhere else.

Trust boundary: the browser only captures; the speech-to-text step uses a free, best-effort service (the one part that isn’t on Databricks); the translation and the final answer both run on Databricks with text only — no audio. A production version would replace the speech step with a hosted speech model so the whole path stays on Databricks. Two tips: the dropdown language must match what you say, and short, clear sentences transcribe most reliably.

6. The technology, in plain terms

Technology	What it does here (plain terms)
Unity Catalog	The governance layer — every table, function and permission lives here, so data is secure and organised.
Medallion pipeline (Delta + Jobs)	The bronze→silver→gold stages that turn raw data into a trustworthy table, re-runnable from scratch.
AI Functions (ai_query)	Reads free text and pulls out structured capabilities; also acts as the auditor that verifies each claim.
Vector Search	A semantic search index over facility text, used to find and compare similar facilities.
Geospatial SQL	Places each facility into the correct district using its map coordinates.
Agent Bricks + Genie	Let the planner ask questions in natural language over the governed data.
Lakebase (Postgres)	A live database that stores saved plans, so a planner’s work persists and can be reopened.
Databricks App (Streamlit)	The hosted web app itself — the 2×2 chart, drill-down, voice box, and save buttons.
Voice + translation	Captures a spoken question in the browser, then translates it to English on Databricks before answering.

Models: the data was built using gemini-3-5-flash; the live app uses Llama-3.3-70B for its answers and translations (the premium models are turned off on the free workspace). Embeddings use gte-large-en.

7. Why you can trust the answer

- **Claims, not facts:** a capability only counts after an AI auditor verifies it.
- **Evidence on demand:** every flag drills down to the facility and the exact source sentence.
- **Honesty banners:** each region shows how much of its data is inferred or low-confidence.

- **Data-poor is never hidden:** a region with no evidence is labelled ‘investigate’, not ‘confirmed gap’.

8. Built to grow

Maternal & newborn care is the live focus because official NFHS-5 data gives a real measure of need. But the engine itself is specialty-agnostic — the same extraction, verification and two-score method works for general surgery, paediatrics, or cardiac care. Adding one is a matter of wiring in that field’s ‘need’ data, not rewriting the system. The app shows those other areas as a clear roadmap, rather than pretending it has data it doesn’t.

9. At a glance

- **Input:** ~10,000 messy India facility records + PIN geography + NFHS-5 survey.
- **Output:** every region scored on care gap × evidence confidence, with verified evidence.
- **Built** entirely on Databricks Free Edition; reproducible from a clean clone.
- **Live** app and public code repository; MIT-licensed.